AI QUALITY PLATFORM

2-Month MVP Execution Plan

Roles · Weekly Tasks · Root Cause Analytics · Pre-ERP Notification

Parameter	Value
Team	3Developer
Capacity	3 × 20 hrs/week × 8 weeks = 480 hrs total
Camera	Arducam UC852 (OV9782) — already connected
Stack	Python 3.10 + OpenCV + FastAPI + SQLite → PostgreSQL
AI	Claude Code terminal + Claude API runtime
MVP Scope	Detect → Root Cause → Pre-ERP Alert → Dashboard
2-Month Verdict	✅ POSSIBLE — with strict scope control

1. ROLE DEFINITIONS

Three roles — one owner per domain. No overlap on core ownership to avoid merge conflicts and confusion.

🔵 jun

Owns: Architecture · CI/CD · Claude integration · Code review · ERP integration · Root cause engine

- Design and maintain overall system architecture
- Write and enforce CI/CD pipeline (GitHub Actions)
- Define all module interfaces before DEV1/DEV2 start coding
- Own claude_helper.py all Claude API calls go through here
- Build root cause analysis engine (Week 5–6)
- Build pre-ERP notification logic (Week 7)
- Unblock DEV1 and DEV2 daily — answer questions same day
- Final code review before every merge to main
- Write and maintain config.yaml — all thresholds live here

Must NOT do:
Write individual detector algorithms (that is marcin) · Write API routes (that is mihao) · Skip code review because it is slow

🟢 Marcin — VISION ENGINEER

Owns: All vision algorithms · Camera pipeline · Test fixtures · Defect classification

- Write capture.py — camera init, frame grab, trigger
- Write preprocessor.py — resize, denoise, normalize
- Write detectors/surface.py — scratch, dust, crack (Canny + morphology)
- Write detectors/color.py — HSV anomaly, coating detection
- Write detectors/brightness.py — over/under exposure
- Write detectors/ensemble.py — combine scores into final PASS/FAIL
- Create all test fixtures — photograph real factory samples
- Write all unit tests for every detector (pytest)
- Use Claude Code to generate first draft of every detector
- Report detection accuracy weekly to CTO

Must NOT do:
Touch API routes · Touch DB schema · Deploy to production — focus 100% on making detection accurate

🟡 Mihao — DATA & API ENGINEER

Owns: Database · FastAPI · Dashboard · Analytics queries · ERP webhook

- Write db/models.py — SQLAlchemy tables (inspection, defect, production_context)
- Write db/repository.py — all CRUD operations
- Write api/main.py — FastAPI app + WebSocket
- Write api/routes/inspection.py — POST /inspect, GET /results
- Write api/routes/analytics.py — root cause query endpoints
- Write api/routes/erp.py — pre-ERP webhook
- Write dashboard/index.html — live operator UI with Chart.js
- Write analytics queries — defect rate by shift, batch, machine state
- Write all API integration tests (pytest)
- Own database migrations when schema changes

Must NOT do:

Write vision algorithms · Change detector thresholds · Modify capture pipeline — focus on data and API layer

2. ROOT CAUSE + PRE-ERP ARCHITECTURE

This is what makes the platform a full service — not just PASS/FAIL detection but understanding WHY quality problems happen and alerting before ERP sees a FAIL.

2.1 Three Layers Beyond Detection

Layer	What It Does	Who Builds	Week
Detection	PASS / FAIL per part in real-time	Marcin	W1–W4
Root Cause Analysis	WHY did defects happen — correlates defect rate with shift, batch, time, environment	Jun	W5–W6
Pre-ERP Alert	Predicts quality degradation BEFORE batch released — sends warning to ERP before FAIL rate spikes	Jun	W7

2.2 Root Cause Data Model

The key insight: every FAIL result must be tagged with production context at the time of inspection. Without context, you cannot find root cause.

```
# db/models.py — extended for root cause

class InspectionResult(Base):
    __tablename__ = 'inspection_results'
    id              = Column(Integer, primary_key=True)
    timestamp       = Column(DateTime, default=datetime.utcnow)
    result          = Column(String)    # 'PASS' | 'FAIL' | 'WARN'
    confidence      = Column(Float)     # 0.0 - 1.0
    defect_types    = Column(JSON)      # ['scratch', 'dust']
    defect_score    = Column(Float)
    frame_path      = Column(String)    # archived image

    # ── Production context (root cause variables) ──
    shift_id        = Column(String)    # 'morning' | 'evening' | 'night'
    operator_id     = Column(String)    # who was operating line
    batch_id        = Column(String)    # material batch / lot number
    line_speed_mpm  = Column(Float)     # conveyor speed at time of inspection
    ambient_temp_c  = Column(Float)     # optional: env sensor
    hour_of_day     = Column(Integer)   # 0-23
    parts_since_last_clean = Column(Integer)  # maintenance correlation

class DefectRegion(Base):
    __tablename__ = 'defect_regions'
    id              = Column(Integer, primary_key=True)
    inspection_id   = Column(Integer, ForeignKey('inspection_results.id'))
    defect_type     = Column(String)
    x = Column(Integer)
    y = Column(Integer)
    w = Column(Integer)
    h = Column(Integer)
    confidence      = Column(Float)

class ProductionContext(Base):
    __tablename__ = 'production_context'
    id              = Column(Integer, primary_key=True)
    timestamp       = Column(DateTime)
    shift_id        = Column(String)
    batch_id        = Column(String)
    operator_id     = Column(String)
    notes           = Column(Text)    # manual notes from operator
```

2.3 Root Cause Engine

```
# src/analytics/root_cause.py  (CTO owns this)
from sqlalchemy import func, text
from src.db.session import get_db

def defect_rate_by_shift(hours=24) -> dict:
    '''Which shift has highest defect rate?'''
    with get_db() as db:
        rows = db.execute(text('''
            SELECT shift_id,
                   COUNT(*) as total,
                   SUM(CASE WHEN result='FAIL' THEN 1 ELSE 0 END) as fails,
                   ROUND(100.0 * SUM(CASE WHEN result='FAIL' THEN 1 ELSE 0 END) / COUNT(*),
2) as fail_rate
            FROM inspection_results
            WHERE timestamp >= datetime('now', :hours)
            GROUP BY shift_id ORDER BY fail_rate DESC
        '''), {'hours': f'-{hours} hours'}).fetchall()
    return [dict(r) for r in rows]

def defect_rate_by_batch() -> dict:
    '''Which material batch causes most defects?'''
    # same pattern — group by batch_id

def defect_trend_alert(window_minutes=30, threshold=0.15) -> dict:
    '''
    PRE-ERP ALERT: if defect rate in last N minutes exceeds threshold,
    return alert before ERP sees a batch FAIL.
    '''
    with get_db() as db:
        recent = db.execute(text('''
            SELECT COUNT(*) as total,
                   SUM(CASE WHEN result='FAIL' THEN 1 ELSE 0 END) as fails
            FROM inspection_results
            WHERE timestamp >= datetime('now', :mins)
        '''), {'mins': f'-{window_minutes} minutes'}).fetchone()
    fail_rate = recent.fails / recent.total if recent.total > 0 else 0
    if fail_rate >= threshold:
        return {
            'alert': True,
            'fail_rate': fail_rate,
            'message': f'Defect rate {fail_rate:.1%} in last {window_minutes}min — exceeds
{threshold:.0%} threshold',
            'action': 'HOLD_BATCH'  # signal to ERP
        }
    return {'alert': False, 'fail_rate': fail_rate}

def generate_root_cause_report(stats: dict) -> str:
    '''Ask Claude to interpret the analytics and suggest root cause.'''
    from src.utils.claude_helper import analyze_root_cause
    return analyze_root_cause(stats)
```

2.4 Pre-ERP Notification Flow

```
# Runs every 5 minutes as background task in FastAPI

# api/main.py
@app.on_event('startup')
async def start_background_tasks():
    asyncio.create_task(pre_erp_monitor())

async def pre_erp_monitor():
    while True:
```

```
        alert = defect_trend_alert(window_minutes=30, threshold=0.15)
        if alert['alert']:
            # 1. Send to ERP BEFORE batch is released
            await send_erp_webhook({
                'event': 'QUALITY_WARNING',
                'fail_rate': alert['fail_rate'],
                'action': 'HOLD_BATCH',
                'message': alert['message']
            })
            # 2. Push to dashboard immediately
            await ws_manager.broadcast({'type': 'ALERT', **alert})
            # 3. Ask Claude for root cause
            stats = get_recent_stats()
            cause = generate_root_cause_report(stats)
            await ws_manager.broadcast({'type': 'ROOT_CAUSE', 'analysis': cause})
        await asyncio.sleep(300)  # check every 5 minutes
```

💡 This is the key differentiator vs simple inspection systems — you catch quality problems 30 minutes before ERP does and explain why.

3. 8-WEEK PLAN — TASK BY ROLE

Color coding: 🔵 Jun 🟢 Marcin (Vision) 🟡 Mihao(Data/API)

WEEK 1 — Dev Environment + Camera Live

Goal: All 3 people have working environment, camera outputs frames, GitHub repo ready

Role	Task	Deliverable
🔵 Jun	Create GitHub repo · Set up branch strategy (main/develop) · Write config.yaml skeleton · Define all module interfaces as empty files · Set up CI (run pytest on push)	Repo live · CI green · All empty files committed
🟢 marcin	Install OpenCV, numpy, pytest · Connect Arducam UC852 · Run test_camera.py · Take 20 sample photos from any object for fixture baseline	Camera outputs 1280×800 color · 20 fixture images saved
🟡 mihao	Install FastAPI, SQLAlchemy, SQLite browser · Study db/models.py interface · Set up local PostgreSQL for later · Write first Pydantic schema for InspectionResult	SQLite DB created · Schema file committed

WEEK 2 — Vision Pipeline Core

Goal: Frames flow through capture → preprocess → ready for detection

Role	Task	Deliverable
🔵 jun	Write vision_bridge.py interface · Define DetectionResult dataclass · Write config loader (config.py) · Review DEV1 + DEV2 code · First Claude Code session: generate capture.py	vision_bridge.py merged · Config loading works
🟢 marcin	Write capture.py using Claude Code · Write preprocessor.py · Write unit tests for both · Measure preprocess time (target < 10ms)	capture.py + preprocessor.py with passing tests
🟡 mihao	Write db/models.py (with production context fields) · Write db/session.py · Write db/repository.py basic CRUD · Write unit tests	DB creates tables · Can insert + query InspectionResult

WEEK 3 — Surface Defect Detector

Goal: First working detector — scratch and dust detected on test images

Role	Task	Deliverable
🔵 jun	Write base Detector abstract class · Define ensemble interface · Review surface.py · Claude prompt: tune thresholds from DEV1 test results	base.py merged · Ensemble interface defined
🟢 marcin	Write detectors/surface.py (Canny + morphological) using Claude Code · Create 10 fixture images (5 PASS + 5 FAIL scratch/dust) · Write tests · Measure accuracy	surface.py: ≥ 80% accuracy on fixtures · Tests pass
🟡 mihao	Write api/main.py skeleton (FastAPI) · Write api/routes/inspection.py (POST /inspect stub) · Write api/schemas.py · Basic API test	FastAPI starts · POST /inspect returns 200

WEEK 4 — Color + Ensemble + First Full Loop

Goal: All detectors working, full pipeline from camera to DB

Role	Task	Deliverable
🔵 jun	Write ensemble.py (weighted score aggregation) · Write decision/engine.py (PASS/FAIL/WARN) · Integration test: camera → detect → decide → DB · Code review sprint	Full pipeline integration test passes
🟢 marcin	Write detectors/color.py (HSV anomaly) · Write detectors/brightness.py · Add 10 more fixtures (color + exposure defects) · Integration test with ensemble	All detectors pass · Ensemble outputs 0.0–1.0 score
🟡 mihao	Connect POST /inspect to real detection pipeline · Save result to DB · Write GET /results endpoint · Add shift_id + batch_id to request schema · API integration tests	End-to-end: inspect → DB saved → query works

WEEK 5 — Root Cause Engine

Goal: Analytics layer built — defect rate by shift/batch queryable

Role	Task	Deliverable
🔵 CTO	Write src/analytics/root_cause.py (all functions from Section 2.3) · Write defect_trend_alert() · Claude integration: generate_root_cause_report() · Unit tests for all analytics	root_cause.py with tests merged
🟢 DEV1	Refine detector thresholds based on real data from Week 4 loop · Add confidence score per defect region · Photograph 20 more factory-realistic fixtures · Help jun test root cause with real data	Improved accuracy · More fixture coverage
🟡 DEV2	Write api/routes/analytics.py (GET /analytics/by-shift, GET /analytics/by-batch, GET /analytics/trend) · Add ProductionContext table · Write analytics API tests	Analytics endpoints return real data

WEEK 6 — Live Dashboard + WebSocket

Goal: Browser shows real-time PASS/FAIL feed + root cause panel

Role	Task	Deliverable
🔵 CTO	Add WebSocket manager to FastAPI · Background task: run inspection loop · Push every result + alerts via WebSocket · Code review all Week 5 code	WebSocket pushes results live
🟢 DEV1	Performance test: measure full pipeline time (target < 100ms) · Fix any bottlenecks · Add WARN state (borderline confidence 0.5–0.7) · Update tests	Pipeline < 100ms confirmed · WARN state works
🟡 DEV2	Write dashboard/index.html with: live feed table + PASS/FAIL chart + defect type breakdown + root cause panel (shows Claude analysis) · WebSocket client JS	Dashboard live in browser · Root cause panel shows

WEEK 7 — Pre-ERP Alert + Claude Reports

Goal: System warns ERP before batch fails — automated shift reports

Role	Task	Deliverable
🔵 CTO	Write pre_erp_monitor() background task · Write send_erp_webhook() · Write claude_helper.analyze_root_cause() · Test with mock ERP server (Flask) · Tune alert threshold	Pre-ERP alert fires correctly · Mock ERP receives it
🟢 DEV1	Run 2-hour continuous inspection loop · Record all false positives · Ask Claude to tune thresholds · Aim for < 5% false positive rate · Document final threshold values in config.yaml	FP rate < 5% · Final config.yaml committed
🟡 DEV2	Add alert panel to dashboard (flashes red on WARN/HOLD_BATCH) · Add GET /report/shift endpoint (returns Claude-generated report) · Add report download button to UI	Alert visible in UI · Shift report downloadable

WEEK 8 — Production Hardening + CI/CD Final

Goal: 4-hour soak test passes · CI green · System deployed on factory PC

Role	Task	Deliverable
🔵 CTO	Complete GitHub Actions CI/CD (C++ + Python + lint) · Add error recovery (auto-restart on crash) · Final architecture review · Write README · Migrate to PostgreSQL	CI green · README complete · PostgreSQL working
🟢 DEV1	Run 4-hour continuous inspection · Log all errors · Fix all crashes · Final fixture set (30+ images) · Submit final accuracy report	4-hour run completes · Accuracy ≥ 85%
🟡 DEV2	Deploy to factory PC · Test all API endpoints on real network · Stress test DB with 1000 records · Final dashboard polish · End-to-end system test	Deployed · All API tests pass on factory PC

💡 jun carries more hours in W5 and W7 (root cause + pre-ERP). mihao carries more in W6 (dashboard). marcin lightens in W7 — accuracy should be done by then.

5. WHAT THE FACTORY SEES AT WEEK 8

Feature	Description	Status at W8
Live inspection feed	Every part inspected in real-time as it passes camera	✅ Done
PASS / FAIL / WARN	Three-state result per part with confidence score	✅ Done
Defect image archive	Every FAIL frame saved with bounding box overlay	✅ Done
Root cause dashboard	Defect rate by shift · by batch · by hour of day	✅ Done
Pre-ERP alert	System warns ERP 30 min before batch fail rate spikes	✅ Done
Claude root cause report	Auto-generated explanation: 'Evening shift has 3× higher scratch rate'	✅ Done
Shift PDF report	Downloadable quality summary per shift	✅ Done
ERP webhook	Sends QUALITY_WARNING + HOLD_BATCH signal to ERP	✅ Done
CI/CD pipeline	Every code push auto-tested — no regressions	✅ Done
Primer UV detection	UV fluorescence coating check	❌ Phase 2
PLC ejector	Auto-reject FAIL parts from conveyor	❌ Phase 2
X-ray internal check	Internal crack + foreign object detection	❌ Phase 3

⚠️ **Phase 2 and Phase 3 features must not be started during the 2-month MVP. Add them to a backlog — do not scope creep.**